Sprint 1

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Grupo 2:

Alejandro Alcázar Mendoza
Sergio Fernández Cordero
Liam Esgueva González

Índice

1. Introducción **3**
1.1. Contexto . . . 3
1.2. Descripción del problema y objetivos del sprint . . . 3
1.3. Justificación . . . 4

2. Metodología **4**
2.1. Entorno de desarrollo . . . 4
2.2. Metodología SCRUM . . . 4
2.3. Diseño de la solución: tareas iniciales . . . 7
2.3.1. Descarga de datos historicos . . . 7
2.3.2. Organización del repositorio y entornos de desarrollo . . . 8
2.3.3. Diseño conceptual de la marca . . . 10
2.4. Diseño de la solución: tareas de HU-2 . . . 13
2.4.1. Exploración de datos iniciales . . . 13
2.4.2. Desarrollo de código para los buckets de AWS . . . 17
2.4.3. Desarrollo de código para subir los archivos a AWS . . . 17
2.5. Pruebas realizadas . . . 19

3. Resultados **19**
3.1. Descripción . . . 19
3.2. Ejecución . . . 19
3.3. Discusión de los resultados . . . 19

4. Conclusión **19**
4.1. Resumen del proceso . . . 19
4.2. Principales logros . . . 19

Sprint 1

1. Introducción

1.1. Contexto

El presente proyecto se desarrolla en el marco de la asignatura Tecnologías de Procesamiento Big Data y tiene como objetivo el diseño e implementación de una arquitectura Big Data completa orientada al análisis de datos de trading de criptomonedas. El proyecto simula un entorno real en el que distintas consultoras tecnológicas trabajan para un gran banco que desea crear un nuevo departamento especializado en el asesoramiento a clientes en trading de criptoactivos.

La solución propuesta se basa en tecnologías de Amazon Web Services (AWS) y contempla tanto el procesamiento de datos históricos en modo batch como la ingesta y análisis de datos en tiempo real (streaming). Para la gestión del proyecto se adopta la metodología ágil Scrum, estructurando el desarrollo en sprints incrementales y entregables funcionales al final de cada iteración.

Este documento recoge la planificación y los objetivos correspondientes al **primer sprint del proyecto**.

1.2. Descripción del problema y objetivos del sprint

Antes de poder abordar el desarrollo técnico de la arquitectura Big Data, es necesario establecer una base sólida que permita garantizar la correcta evolución del proyecto en los siguientes sprints. En este sentido, el problema principal abordado en este primer sprint es la falta de conocimiento inicial del dominio del proyecto, de las fuentes de datos y de la estructura de almacenamiento, así como la necesidad de una organización adecuada del trabajo colaborativo.

El Sprint 1 se centra en la ejecución de las dos primeras historias de usuario definidas para el proyecto, que persiguen familiarizar al equipo con el contexto, las fuentes de datos y sentar las bases de la capa de almacenamiento. Los objetivos del sprint son los siguientes:

- **Completar la Historia de Usuario HU-1**, adquiriendo una comprensión global del proyecto, de la herramienta TradingView y de las fuentes de datos históricos de criptomonedas.
- **Completar la Historia de Usuario HU-2**, diseñando una primera organización de los datos históricos y creando los buckets necesarios en Amazon S3.
- **Organizar y estructurar el repositorio del proyecto**, estableciendo una base clara para el desarrollo futuro.
- **Orientar el diseño de la marca del proyecto**, incluyendo los primeros bocetos y lineamientos visuales que se desarrollarán en sprints posteriores.
- **Desarrollar una base** sólida que permita desarrollar los siguientes sprints.

1.3. Justificación

La ejecución de este primer sprint resulta fundamental para **reducir riesgos técnicos y organizativos en fases posteriores del proyecto**. Al dedicar una iteración completa a comprender el dominio del problema, las fuentes de datos y la arquitectura inicial, se evita la toma de decisiones prematuras que podrían derivar en retrabajo en sprints futuros.

Asimismo, la definición temprana de la estructura del repositorio y de la organización del almacenamiento en Amazon S3 permite **establecer buenas prácticas desde el inicio**, facilitando el trabajo colaborativo del equipo y la escalabilidad de la solución Big Data. Finalmente, la incorporación de una orientación inicial del diseño de la marca contribuye a **dotar al proyecto de una identidad coherente desde sus primeras fases**, alineando tanto los aspectos técnicos como los visuales y comunicativos del desarrollo

2. Metodología

2.1. Entorno de desarrollo

Para el desarrollo del proyecto se ha definido un entorno de trabajo común que permita al equipo abordar de forma eficiente las tareas asociadas al procesamiento y análisis de datos Big Data, así como la gestión colaborativa del proyecto.

Como lenguaje principal se empleará **Python**, que será utilizado para la implementación de scripts destinados a la descarga y tratamiento inicial de los datos históricos de criptomonedas. La obtención de los datos se realizará a partir de la plataforma **TradingView**, utilizando fuentes públicas que proporcionan información histórica de precios de criptomonedas. Estos datos serán posteriormente almacenados en **buckets de Amazon S3**, que actuarán como capa inicial de almacenamiento para el procesamiento en modo batch.

Para la gestión del código y el trabajo colaborativo del equipo se utilizará **GitHub**, donde se organizará el repositorio del proyecto siguiendo una estructura clara y escalable. En dicho repositorio se incluirá un archivo requirements.txt que permitirá definir y reproducir de forma sencilla el entorno virtual de Python, garantizando la consistencia del entorno de ejecución entre los distintos miembros del equipo.

Adicionalmente, se emplearán herramientas de diseño gráfico y visual para la creación de los primeros elementos visuales del proyecto. Estos elementos servirán como base para la identidad visual del proyecto y para orientar el diseño de las visualizaciones que se desarrollarán en sprints posteriores.

2.2. Metodología SCRUM

Para la gestión y desarrollo del proyecto se adoptará la metodología ágil Scrum, con el objetivo de estructurar el trabajo en iteraciones cortas e incrementales que permitan una evolución progresiva y controlada de la solución Big Data. Esta metodología facilita la adaptación a cambios, la entrega continua de valor y la validación temprana de los resultados obtenidos en cada fase del proyecto.

Historia de Usuario (HU-1)

1.1. Como:
Manager del proyecto.

1.2. Quiero:
Que las consultoras:

- Conozcan el proyecto
- Conozcan la herramienta TradingView
- Conozcan sus fuentes de datos históricos:
- https://github.com/ravalmeet/TradingView-Data/tree/master Este repositorio recoge la forma de obtener el histórico de los datos. La propuesta inicial es obtener 4 años con una frecuencia de 1 día
- Conjunto de 13 criptomonedas objeto del proyecto con su símbolo. Se recomienda utilizar el Exchange BINANCE para la obtención de los datos. En el caso de este grupo, la criptomoneda asignada es **Cardano (ADAUSD)**.

1.3. Para:
Poder empezar con garantías el proyecto.

Las tareas asociadas a la **Historia de Usuario HU-1** serán realizadas **de forma conjunta por todos los miembros del equipo**. Dado que esta historia de usuario constituye la base conceptual y técnica del proyecto, su ejecución requiere una comprensión compartida del contexto, de las fuentes de datos y de las herramientas que se utilizarán a lo largo de los distintos sprints.

El trabajo colaborativo en esta fase inicial permite alinear criterios, reducir posibles desviaciones en etapas posteriores y garantizar que todos los integrantes del equipo parten de un conocimiento común, lo que resulta esencial para el correcto desarrollo del proyecto.

Historia de Usuario (HU-2)

2.1. Como:
Manager del proyecto.

2.2. Quiero:
Que las consultoras:

- Diseñen una primera organización de la información histórica basada en buckets y una jerarquía de carpetas
- Creen el/los buckets necesarios en Amazon S3
- Definan el formato de los archivos de datos históricos que se van a almacenar en Amazon S3
  - Deben cargar todos los datos posibles, para que no tengan que volver a atrás en los sprint.
  - Deben hacerlo lo más genérico posible
- Suban los primeros archivos en formato CSV

2.3. Para:
Empezar a definir la capa de almacenamiento de datos históricos.

Como resultado del trabajo de toma de contacto realizado en la Historia de Usuario HU-1, se han definido una serie de **tareas iniciales que actúan como punto intermedio** entre la fase de comprensión del proyecto y el inicio del desarrollo técnico en la nube, que se abordará de manera más directa en la Historia de Usuario HU-2. Estas tareas permiten comenzar a materializar el proyecto sin entrar aún de lleno en la infraestructura de AWS, sirviendo como preparación técnica y organizativa necesaria para los siguientes sprints. Las tareas definidas para esta fase son las siguientes.

Tareas iniciales individuales

- **Desarrollo del código para la descarga de los datos históricos**: utilización del lenguaje Python con el objetivo de validar el acceso a las fuentes de datos y preparar la información que será posteriormente almacenada en Amazon S3. *Responsable:* ***Liam***.
- **Organización del repositorio y preparación de los entornos de desarrollo**: definición de la estructura del proyecto en GitHub y configuración del entorno virtual mediante el archivo `requirements.txt`, garantizando la consistencia del entorno de ejecución. *Responsable:* ***Sergio***.
- **Diseño conceptual de la marca del proyecto**: elaboración de los primeros lineamientos visuales, identidad gráfica y bocetos iniciales que servirán como base para las futuras visualizaciones y dashboards. *Responsable:* ***Alejandro***.

Una vez completadas las tareas iniciales del Sprint 1 y establecida la base técnica y organizativa del proyecto, se procede a la distribución de **tareas asociadas a la Historia de Usuario HU-2**. En esta fase, el equipo comienza el desarrollo efectivo de la capa de almacenamiento en AWS, asignando responsabilidades específicas a cada miembro con el objetivo de avanzar de forma coordinada y eficiente en la definición de la arquitectura Big Data.

Tareas de la segunda historia

- **Exploración de datos iniciales**: análisis preliminar de los datos históricos descargados con el objetivo de validar su calidad, estructura y coherencia antes de su procesamiento posterior. *Responsable: Liam.*
- **Desarrollo de código para la creación de los buckets en AWS**: implementación de scripts que permiten crear y configurar de forma programática los buckets necesarios en Amazon S3, siguiendo la estructura definida para el almacenamiento de datos históricos. *Responsable: Alejandro.*
- **Desarrollo de código para subir los archivos a los buckets**: desarrollo de scripts en Python para automatizar la carga de los archivos CSV generados en los buckets de Amazon S3, garantizando la correcta organización de los datos. *Responsable: Sergio.*

Durante la ejecución de estas tareas, el seguimiento del trabajo se realizará mediante la creación y cierre de **issues** en el repositorio de GitHub, lo que permitirá registrar el progreso de cada actividad, facilitar la coordinación entre los miembros del equipo y mantener un control claro del estado de las tareas asociadas.

2.3. Diseño de la solución: tareas iniciales

En este apartado se describe el diseño de la solución a partir de las tareas iniciales definidas en el primer sprint del proyecto.

2.3.1. Descarga de datos historicos

2.3.1.1. Motivación y diseño orientado a objetos

La decisión de estructurar la funcionalidad de carga de datos mediante una clase responde a varios principios de ingeniería de software. En primer lugar, la programación orientada a objetos permite encapsular el estado y comportamiento relacionados con la obtención de datos en una única entidad cohesiva. Esto facilita la reutilización del código en diferentes partes del proyecto y permite mantener una separación clara de responsabilidades. La clase `CryptoDataLoader` actúa como una interfaz unificada que abstrae la complejidad de interactuar con APIs externas, procesar respuestas y gestionar el almacenamiento de archivos.

Además, este enfoque modular permite modificar la implementación interna sin afectar al resto del sistema, siguiendo el principio de bajo acoplamiento. Por ejemplo, si en el futuro se decide cambiar el proveedor de datos o el formato de almacenamiento, solo sería necesario modificar esta clase sin impactar en los componentes que la utilizan.

2.3.1.2. Arquitectura y flujo de ejecución

El flujo de trabajo de la clase `CryptoDataLoader` sigue una secuencia estructurada que garantiza la integridad y consistencia de los datos. El proceso comienza con la inicialización del objeto, donde se especifican los parámetros esenciales: el símbolo del par de trading (en este caso, `ADAUSDT`), el exchange de origen (`BINANCE`), el intervalo temporal deseado y el directorio de salida para los archivos generados.

Una vez inicializada la instancia, el método `download_history` se encarga de establecer la comunicación con la API de TradingView a través de la biblioteca `trading_view_data`. Este método realiza una petición especificando el número de barras o registros a recuperar (parámetro `n_bars`), que en la configuración final se estableció en 1483 barras para obtener datos diarios desde enero de 2022 hasta la actualidad.

La respuesta de la API se recibe en forma de DataFrame de pandas, que contiene las columnas estándar de datos OHLCV (Open, High, Low, Close, Volume) junto con la marca temporal correspondiente. Este DataFrame se somete a una validación básica para asegurar que no está vacío y que contiene datos válidos.

Posteriormente, el método `split_and_save_by_month` procesa el DataFrame mediante una operación de agrupación temporal. Utilizando expresiones regulares, se extrae el año y mes de cada registro a través del método auxiliar `_extract_year_month`, que analiza el formato de fecha y genera una clave en formato `YYYY_MM`. Los datos se agrupan según esta clave y se almacenan en un diccionario donde cada entrada corresponde a un mes específico de datos históricos.

Finalmente, el método `run` orquesta todo el pipeline ejecutando secuencialmente las operaciones de descarga, división y almacenamiento, proporcionando una interfaz simple para ejecutar todo el proceso con una única llamada.

2.3.1.3. Proceso de exploración y experimentación

La configuración final del sistema de carga de datos es el resultado de un extenso proceso de exploración y experimentación que involucró múltiples iteraciones y ajustes. Inicialmente, se llevaron a cabo pruebas en entornos de Jupyter Notebook para validar diferentes configuraciones de manera interactiva y visual.

Durante esta fase exploratoria, se experimentó con diversos intervalos temporales para determinar cuál proporcionaría el óptimo para el análisis. Se probaron configuraciones con datos de minuto, hora, día y semana, evaluando en cada caso la disponibilidad histórica y la cantidad de registros obtenidos. Asimismo, se ajustó iterativamente el parámetro `n_bars`, comenzando con valores conservadores de 100 barras para verificar el funcionamiento básico, y aumentando progresivamente hasta alcanzar los valores necesarios para cubrir el rango temporal objetivo.

2.3.1.4. Limitaciones encontradas y alternativas evaluadas

Durante el desarrollo del proyecto se detectó una limitación crítica en la disponibilidad de datos históricos horarios para el par ADAUSDT en Binance, ya que la API de TradingView solo ofrecía información desde 2025, un período insuficiente para el análisis planteado inicialmente. La estrategia original consistía en utilizar datos horarios para generar agregaciones diarias y mensuales y así construir características multi-temporales para el modelo.

Se evaluaron fuentes alternativas como Coinbase y la biblioteca `yfinance`, pero ambas presentaron restricciones similares en cuanto a profundidad histórica de datos de alta frecuencia. Ante estas limitaciones, se decidió adaptar el enfoque y trabajar exclusivamente con datos diarios, que sí ofrecían un historial amplio desde enero de 2022.

Esta situación evidencia las dificultades habituales en proyectos con criptomonedas, donde las políticas de retención y las limitaciones técnicas de las APIs obligan a rediseñar estrategias y mantener flexibilidad en el desarrollo de sistemas de trading algorítmico.

2.3.2. Organización del repositorio y entornos de desarrollo

La estructura del repositorio del proyecto se ha organizado siguiendo convenciones estándar de desarrollo en Python, facilitando la navegabilidad y el mantenimiento del código. En la raíz del proyecto se encuentra el directorio principal `ADAQuant`, que contiene los módulos core del sistema, incluyendo los scripts de carga de datos (`crypto_data_loader.py`, `trading_view_data.py`), la configuración de buckets de almacenamiento (`create_buckets.py`) y los archivos de inicialización de paquete.

En cuanto al entorno de desarrollo, el proyecto utiliza dos entornos de desarrollo diferenciados. Por un lado, se ha decidido mantener una separación clara entre el sistema operativo del equipo y las dependencias específicas del proyecto. Con este objetivo, se ha creado un entorno virtual de Python denominado `ADAQuantEnv`, que permite aislar las librerías y configuraciones necesarias para la ejecución del código desarrollado.

El entorno virtual ha sido creado utilizando la herramienta `uv` y WSL, lo que proporciona una gestión eficiente y reproducible de las dependencias, así como la posibilidad de seleccionar de forma explícita la versión de Python utilizada en el proyecto. Este enfoque facilita la consistencia del entorno de ejecución entre los distintos miembros del equipo y reduce la probabilidad de conflictos derivados de diferencias en configuraciones locales.

El uso de un entorno virtual aislado aporta, además, una mayor seguridad y estabilidad al desarrollo, ya que cualquier error o modificación en las dependencias del proyecto queda restringido al entorno virtual, evitando fallos sobre el sistema operativo o sobre otros proyectos. De este modo, se garantiza una base sólida y controlada para el desarrollo incremental del proyecto en los siguientes sprints.

Por otro lado, el directorio `TradingView-Data` contiene notebooks de Jupyter utilizados durante la fase de exploración y experimentación inicial. Estos notebooks (`TradingviewData.ipynb` y su copia de trabajo) sirvieron como entorno interactivo para probar diferentes configuraciones de extracción de datos, validar la conectividad con las APIs y visualizar los resultados obtenidos antes de consolidar la funcionalidad en los módulos de Python definitivos.

Adicionalmente, el repositorio incluye directorios auxiliares como `load_data` para almacenamiento temporal, `brand_visuals` para recursos gráficos del proyecto, y `local` para configuraciones específicas del entorno de desarrollo, junto con archivos estándar de documentación y control de versiones.

2.3.3. Diseño conceptual de la marca

El proceso de diseño del logotipo comenzó con la exploración de las letras principales que componen el nombre de la marca, centrando el trabajo en la **A y la Q** como elementos distintivos. En una primera fase se partió de una caligrafía más rígida y estructurada, buscando formas claras y reconocibles que aportasen solidez y seriedad al proyecto. Sin embargo, tras varias iteraciones y bocetos, se consideró que este enfoque resultaba excesivamente estático para un proyecto relacionado con el análisis de datos financieros y mercados dinámicos como el de las criptomonedas, por lo que se optó por un diseño más fluido. Finalmente se diseñó un boceto para el logotipo de la marca.



Figura 1. Evolución del diseño: A



Figura 2. Evolución del diseño: Q



Figura 3. Boceto del logotipo

Una vez definidas las versiones finales, se emplearon **herramientas de inteligencia artificial** como apoyo en la generación del logotipo final. La IA se utilizó como una herramienta complementaria al proceso creativo, permitiendo explorar distintas combinaciones, ajustes formales y acabados visuales

a partir de los diseños previamente definidos de forma manual. Este enfoque híbrido, que combina trabajo manual e inteligencia artificial, permitió acelerar el proceso de diseño sin perder coherencia con la identidad visual planteada, dando como resultado un logotipo alineado con los valores y objetivos del proyecto.



Figura 4. Logotipo ADAQuant

Como elemento complementario al logotipo y a la identidad visual de la marca, se diseñó una mascota con el objetivo de reforzar el carácter del proyecto y facilitar su aplicación en distintos contextos visuales, como dashboards, presentaciones o material divulgativo. La mascota elegida es un **rinoceronte**, un animal que simboliza solidez, resistencia y determinación.

El proceso de diseño comenzó a partir de un boceto realizado a mano, en el que se definieron las líneas generales, la actitud del personaje y su estilo visual. Posteriormente, este boceto inicial se utilizó como referencia para el uso de herramientas de inteligencia artificial para ajustar proporciones, expresividad y nivel de detalle.



Figura 5. Boceto de la mascota

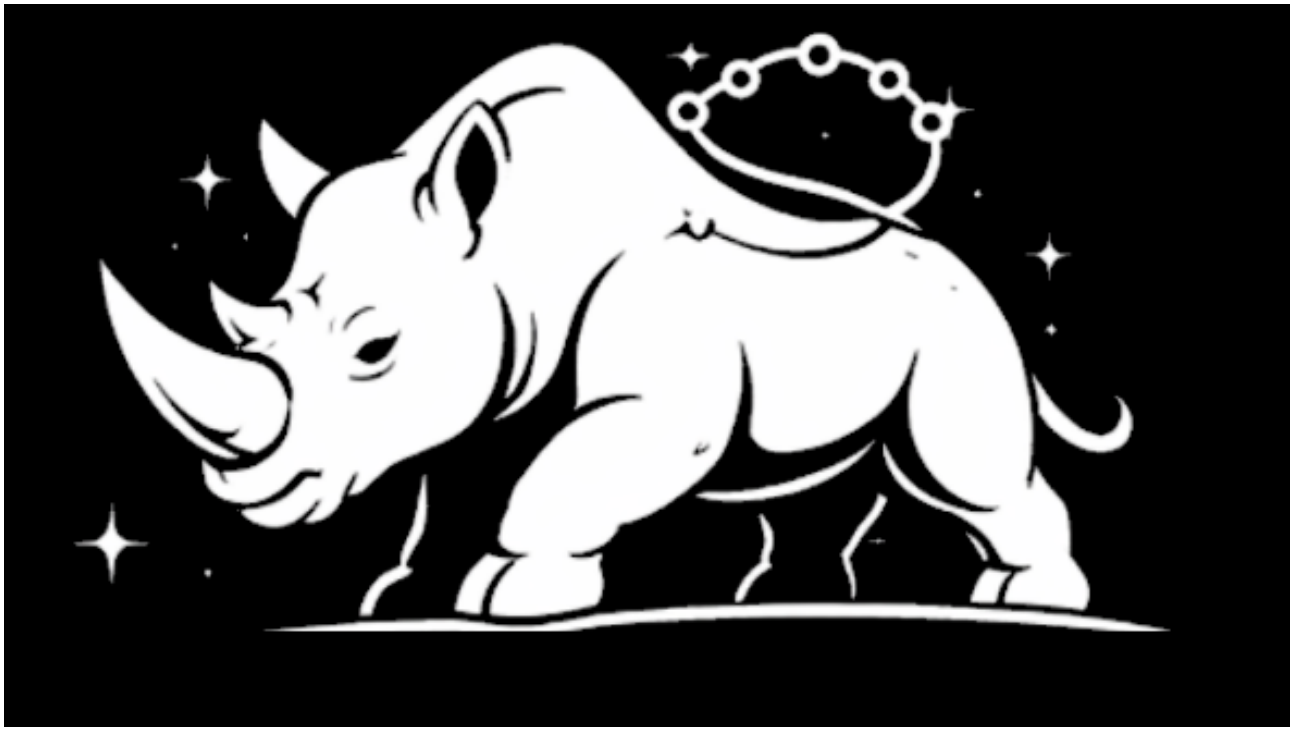


Figura 6. Diseño final

Con estos resultados, se escoge una **paleta de colores** para la barca y se diseña un **banner** para el repositorio o para presentaciones.



Figura 7. Paleta de colores

2.4. Diseño de la solución: tareas de HU-2

Para el desarrollo de las tareas asociadas a la Historia de Usuario HU-2 se ha definido un diseño basado en la creación de tres clases principales, con el objetivo de estructurar y modular el proceso de gestión de los datos históricos. Este enfoque permite separar responsabilidades, mejorar la mantenibilidad del código y facilitar su evolución en sprints posteriores.

La clase **CryptoDataLoader** será la encargada de gestionar la búsqueda y obtención de la información histórica de las criptomonedas a partir de las fuentes definidas, ya comentada en la sección 2.3.1 . Por su parte, la clase **S3YearlyBucketManager** se utilizará para la creación y gestión de los buckets en Amazon S3, siguiendo la estructura de almacenamiento establecida para los datos históricos. Finalmente, la clase **S3DataUploader** se encargará de la subida de los archivos generados a los buckets correspondientes, automatizando el proceso de carga y organización de los datos.

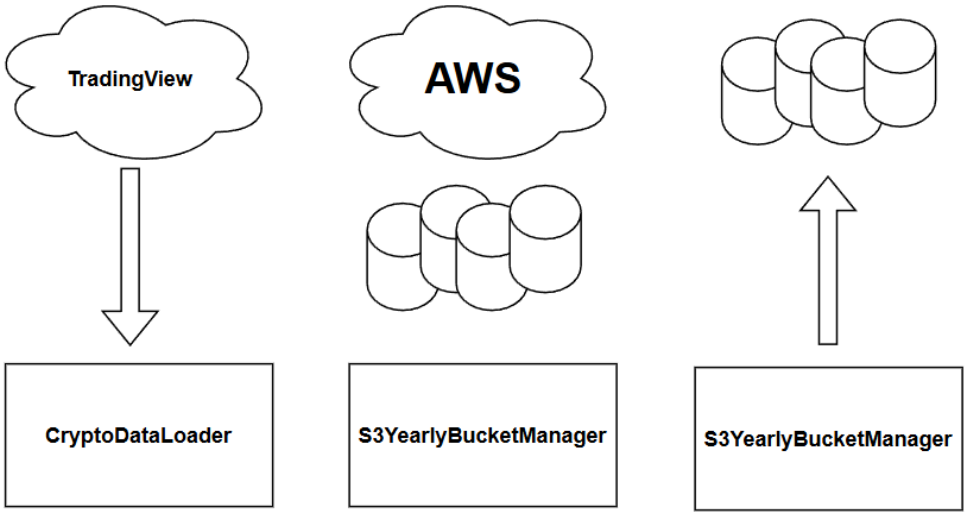


Figura 8. Arquitectura del sprint

2.4.1. Exploración de datos iniciales

Una vez completada la descarga de los datos históricos de Cardano (ADAUSDT), se realizó un análisis exploratorio para validar la calidad de la información y comprender las características estadísticas del activo. Este análisis constituye la base para el procesamiento posterior y el desarrollo de modelos predictivos en futuros sprints.

2.4.1.1. Análisis de distribuciones y valores atípicos

El análisis univariado mediante boxplots reveló que las variables de precio (open, high, low, close) presentan numerosos valores atípicos en la región superior del rango, reflejando la naturaleza volátil del mercado de criptomonedas. Los histogramas de distribución muestran un fuerte sesgo positivo, con concentración de observaciones en valores bajos y colas largas hacia precios elevados, indicando que Cardano ha permanecido la mayor parte del tiempo en niveles de baja cotización con picos esporádicos de revalorización.

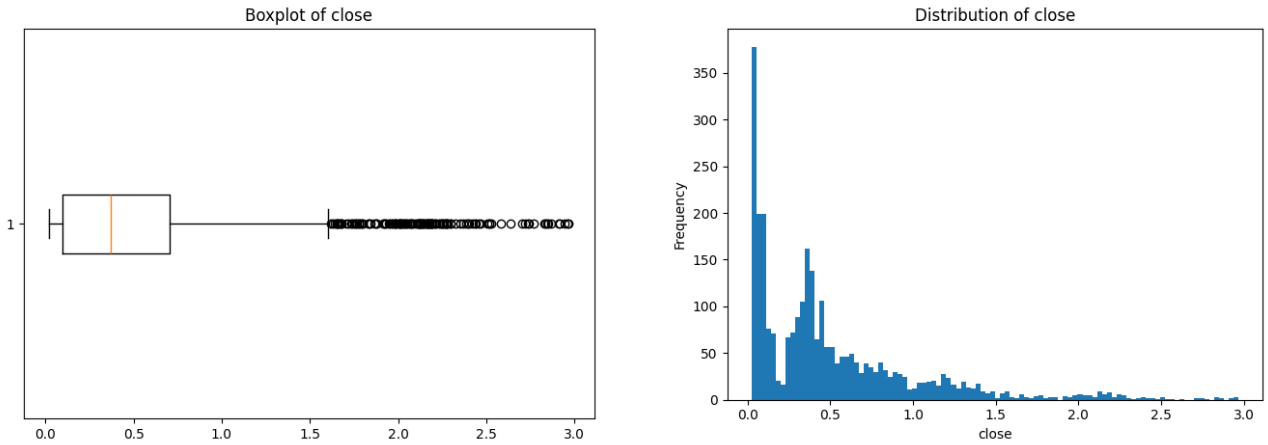


Figura 9. Boxplot y distribución del precio de cierre

El volumen de transacciones presenta un comportamiento diferenciado, con la mayoría de observaciones concentradas en valores muy bajos pero con outliers extremos que alcanzan 2.0×10 unidades, evidenciando períodos de baja actividad interrumpidos por episodios de alta intensidad comercial. Esto ocurrió en la salida de la criptomoneda ya que se presentaba como la solución a los contratos inteligentes, pero esta promesa se desvaneció con el tiempo.

2.4.1.2. Correlaciones entre variables

La matriz de correlación confirma una asociación casi perfecta (>0.99) entre las cuatro variables de precio, mientras que el volumen presenta correlaciones muy bajas (0.2) con el precio. Este resultado indica que los movimientos de precio no están necesariamente acompañados de incrementos proporcionales en la actividad comercial, sugiriendo que otros factores como el sentimiento del mercado o eventos del ecosistema blockchain tienen mayor influencia.

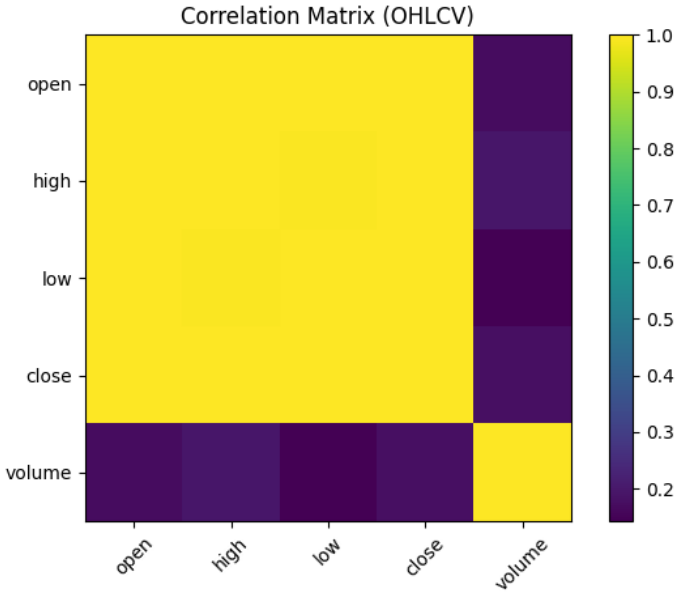


Figura 10. Matriz de correlación entre variables OHLCV

2.4.1.3. Evolución temporal y ciclos de mercado

La serie temporal del precio muestra tres fases diferenciadas: estabilidad en niveles bajos durante 2018-2020 (0.03-0.15 USDT), un ciclo alcista extraordinario en 2021-2022 con pico cercano a 3.0 USDT coincidiendo con el boom general de criptomonedas, y una corrección posterior que sitúa el precio actual en torno a 0.4 USDT. Esta dinámica cíclica es característica de mercados especulativos con alta sensibilidad a factores macroeconómicos.

Cardano subió con fuerza en 2021 porque coincidió con un gran mercado alcista en criptomonedas, abundante liquidez en la economía y altas expectativas por el lanzamiento de los contratos inteligentes. Muchos inversores compraron anticipando un crecimiento futuro del proyecto, lo que empujó el precio muy por encima de su uso real en ese momento.

La caída posterior se produjo cuando terminó ese ciclo: subieron los tipos de interés, se redujo el apetito por el riesgo y ocurrieron grandes colapsos dentro del sector cripto que dañaron la confianza. Además, el desarrollo y adopción de Cardano avanzaron más despacio de lo que el mercado había descontado.

En resumen, el boom vino por euforia y expectativas (se la llegó a conocer como Etherium killer), y la bajada por el cambio macroeconómico, crisis internas del sector y una corrección hacia valoraciones más realistas.

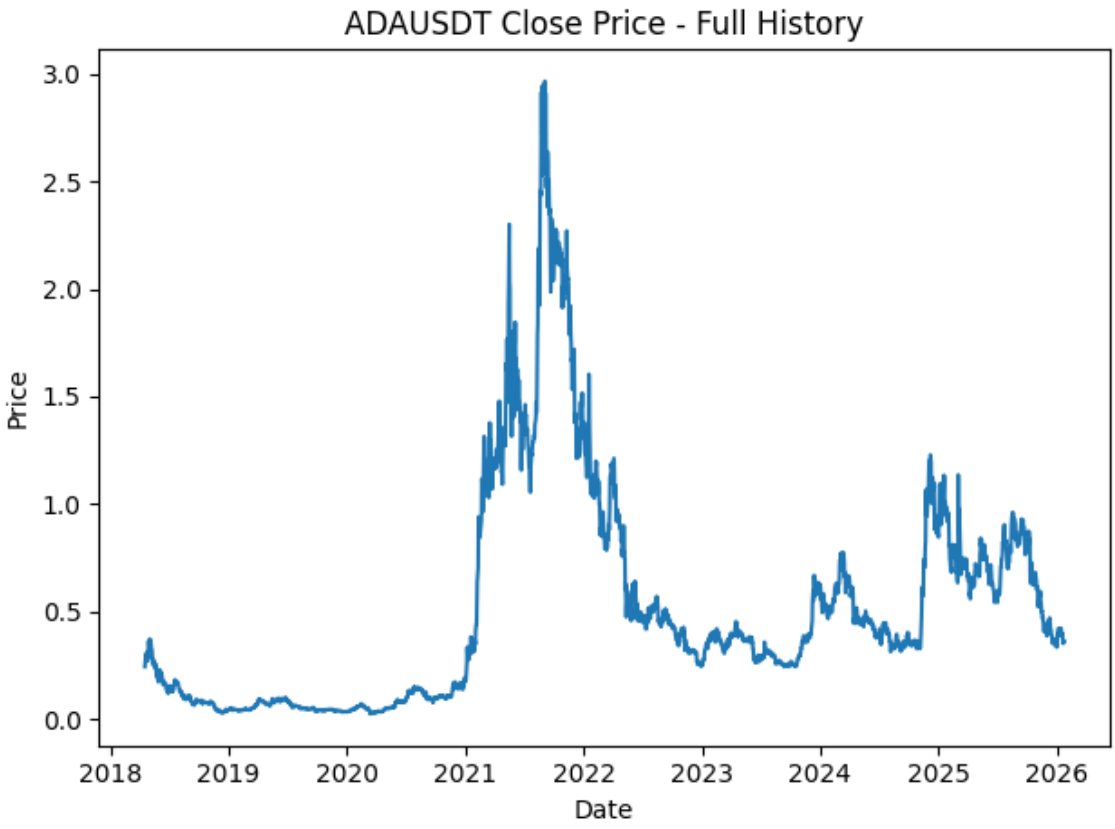


Figura 11. Evolución histórica del precio de cierre (2018-2026)

La gráfica de retornos diarios muestra que la mayoría de los movimientos del precio se concentran en un rango relativamente estrecho, aproximadamente entre 5 y +5 , lo que indica un comportamiento

habitual moderado en condiciones normales de mercado. Sin embargo, aparecen episodios extremos, con subidas o caídas muy pronunciadas, especialmente alrededor de 2021–2022 y de nuevo hacia 2025, lo que refleja momentos de alta tensión y cambios bruscos en el sentimiento de los inversores.

La volatilidad rolling de 30 días confirma este patrón de inestabilidad por etapas. Se observa un claro clustering temporal: periodos prolongados de baja volatilidad seguidos por picos abruptos. Destacan especialmente los repuntes durante 2020–2021 y, con mayor intensidad, en 2025, cuando la volatilidad alcanza sus niveles más altos del período analizado, señalando fases de elevada incertidumbre y riesgo en el mercado.

En conjunto, ambas gráficas indican que el activo suele moverse de forma relativamente estable la mayor parte del tiempo, pero alterna con episodios de estrés extremo concentrados en ciertos años, característicos de los ciclos especulativos y de shocks externos en los mercados de criptomonedas.

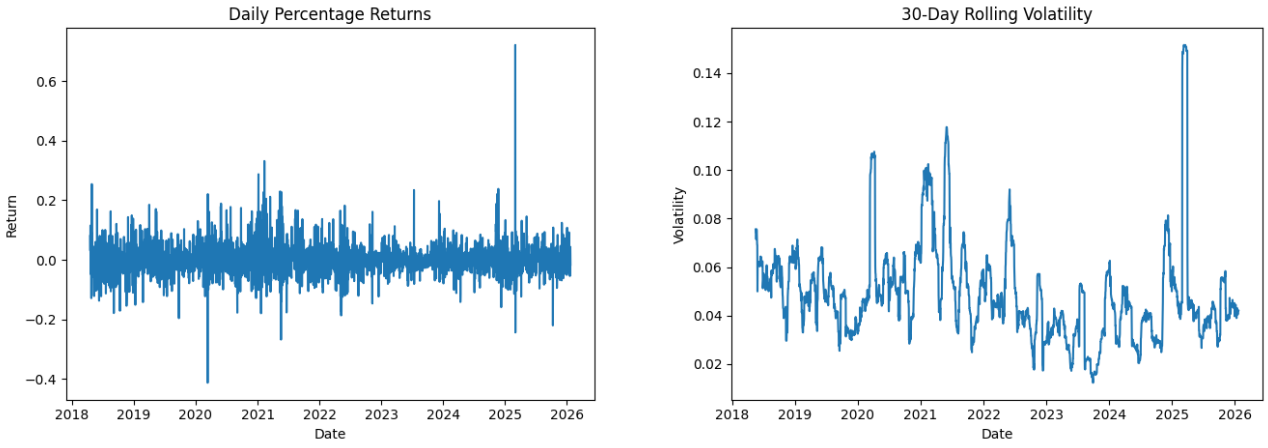


Figura 12. Retornos diarios y volatilidad rolling de 30 días

2.4.1.4. Relación volumen-precio

El gráfico de dispersión confirma visualmente la baja correlación lineal entre volumen y precio, aunque se observa que los precios más elevados raramente ocurren con volúmenes muy bajos, sugiriendo que alcanzar niveles extremos de precio requiere cierta liquidez mínima para sostenerlos.

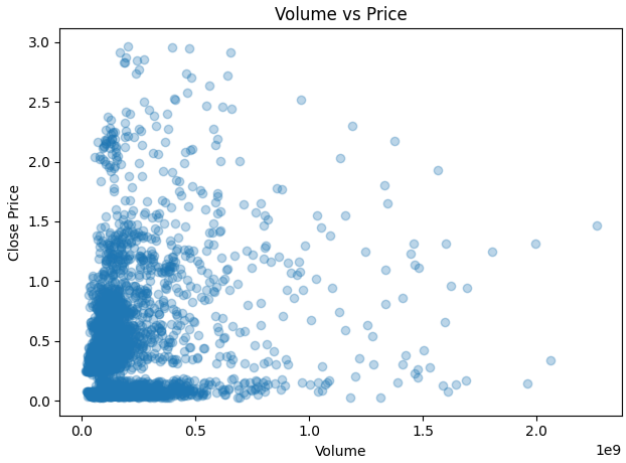


Figura 13. Relación entre volumen y precio de cierre

2.4.1.5. Conclusiones del análisis exploratorio

El análisis confirma la calidad de los datos recopilados y revela características clave del activo: ciclos bien definidos de apreciación-corrección, volatility clustering aprovechable para modelos predictivos, distribuciones con colas pesadas que implican riesgo extremo, y baja correlación volumen-precio que sugiere la necesidad de incorporar variables adicionales en futuros modelos de trading algorítmico.

2.4.2. Desarrollo de código para los buckets de AWS

Para la definición de la capa inicial de almacenamiento de datos históricos se ha desarrollado un módulo en Python que automatiza la creación y configuración de los buckets de Amazon S3 necesarios para el proyecto. Esta implementación permite establecer desde el inicio una estructura coherente, segura y escalable para el almacenamiento de los datos.

La solución se basa en la creación de **buckets anuales**, siguiendo una convención de nombres homogénea formada por un prefijo común y el año correspondiente (por ejemplo, `adaquant-2022`, `adaquant-2023`). Esta organización facilita la segmentación temporal de los datos y simplifica tanto su mantenimiento como su procesamiento posterior en modo *batch*.

Dentro de cada bucket anual se inicializa una **estructura de carpetas basada en meses**, generando prefijos desde `month=01` hasta `month=12`. Esta jerarquía permite una clasificación temporal más granular de los datos históricos y está alineada con buenas prácticas habituales en arquitecturas Big Data.

Además de la creación de los buckets y su estructura interna, la implementación aplica automáticamente una serie de **configuraciones de seguridad y buenas prácticas**, entre las que se incluyen:

- Bloqueo completo del acceso público a los buckets.
- Activación del versionado de objetos, permitiendo mantener un histórico de cambios en los datos.
- Habilitación del cifrado del lado del servidor (SSE-S3) para proteger la información almacenada.

Toda esta lógica se encapsula en una clase encargada de gestionar el ciclo de vida de los buckets, de manera que el resto del proyecto puede crear la infraestructura necesaria simplemente indicando el número de años a generar y el año inicial. Este enfoque modular y automatizado reduce errores de configuración manual y establece una base sólida para el desarrollo posterior del sistema Big Data en AWS.

2.4.3. Desarrollo de código para subir los archivos a AWS

Una vez implementadas las funcionalidades de descarga de datos históricos y de creación de los buckets anuales en Amazon S3, el siguiente paso del Sprint 1 consiste en integrar ambos componentes para almacenar los datos históricos en buckets AWS. Para ello se ha creado una clase encargada de subir los archivos CSV generados a los buckets correspondientes.

El objetivo principal de esta fase es automatizar la carga de los datos históricos mensuales de Cardano (ADAUSDT) en Amazon S3, asegurando que cada conjunto de datos se almacene en el bucket del año correspondiente y dentro de la carpeta asociada al mes, respetando la estructura previamente definida.

2.4.3.1. Integración del flujo de adquisición de datos y creación de buckets

El diseño de esta parte del sistema se basa en la integración directa de los dos componentes desarrollados previamente en el sprint: el módulo de adquisición de datos, encargado de descargar el histórico de precios desde TradingView y generar conjuntos de datos mensuales, y el módulo de creación de buckets, encargado de la creación de los buckets anuales en Amazon S3 junto con su estructura mensual mediante carpetas.

La clase desarrollada para esta tarea actúa como un componente intermedio que recibe los datos descargados y los almacena en la infraestructura de S3 ya creada. Dado que los buckets siguen una organización anual y contienen carpetas mensuales (`month=MM/`), el proceso de subida resulta directo y fácilmente extensible.

Cada archivo CSV generado corresponde a los datos de un mes concreto y se almacena dentro del bucket del año correspondiente, en la carpeta del mes adecuado. Esta organización permite una estructura clara de la información y facilita tanto la verificación manual de los datos como su posterior procesamiento en futuros sprints.

Para verificar la ejecución del proceso, durante la ejecución del script se genera una salida informativa por consola que confirma la subida de cada archivo CSV a Amazon S3. En la Figura 14 se muestra un ejemplo de este mensaje de confirmación.

```
Uploaded adausdt_daily_2023_08.csv to s3://adaquant-2023/month=08/adausdt_daily_2023_08.csv
Uploaded adausdt_daily_2023_09.csv to s3://adaquant-2023/month=09/adausdt_daily_2023_09.csv
Uploaded adausdt_daily_2023_10.csv to s3://adaquant-2023/month=10/adausdt_daily_2023_10.csv
Uploaded adausdt_daily_2023_11.csv to s3://adaquant-2023/month=11/adausdt_daily_2023_11.csv
Uploaded adausdt_daily_2023_12.csv to s3://adaquant-2023/month=12/adausdt_daily_2023_12.csv
Uploaded adausdt_daily_2024_01.csv to s3://adaquant-2024/month=01/adausdt_daily_2024_01.csv
```

Figura 14. Salida por consola confirmando la subida correcta de los archivos CSV a Amazon S3

2.4.3.2. Funciones auxiliares para la gestión del almacenamiento

Con el objetivo de mantener un diseño modular y facilitar el mantenimiento del código, la clase encargada de la subida de datos incorpora varias funciones auxiliares que encapsulan tareas específicas del proceso. Estas funciones permiten separar la lógica del flujo principal y evitar la duplicación de código.

En primer lugar, se crea una función responsable de extraer el año y el mes a partir del id temporal asociado a cada conjunto de datos mensuales, con formato `YYYY_MM`. Con esto, se podrá determinar el bucket de destino y la carpeta interna en la que debe almacenarse el csv.

Además, se define una función dedicada a la construcción del nombre del bucket anual, siguiendo la convención establecida en el proyecto. Centralizar esta lógica en una función permite modificar fácilmente el formato de nombres sin acarrear un error al resto del sistema.

Por último, se realiza una función que construirá la ruta interna del archivo dentro del bucket de Amazon S3, asociada a la carpeta del mes correspondiente y al nombre final del archivo CSV. Igual que antes, esta separación de responsabilidades permite dar consistencia a la estructura de almacenamiento y a su modificación.

2.4.3.3. Flujo completo de subida de los datos históricos

El flujo completo de subida de los datos históricos se implementa en un método principal que coordina el uso de las funciones auxiliares y la operación de carga a Amazon S3. Este método toma como entrada un diccionario que contiene los DataFrames mensuales generados durante la fase de adquisición de datos.

Para cada conjunto de datos mensual, el proceso sigue los siguientes pasos: se extrae el año y el mes del identificador temporal, se determina el bucket anual de destino, se construye la ruta interna del archivo dentro del bucket y, finalmente, el DataFrame se convierte a formato CSV en memoria y se sube directamente a Amazon S3.

Cabe destacar que no se realiza ninguna comprobación explícita de duplicados durante la subida de archivos. Dado que los buckets de Amazon S3 tienen habilitado el versionado de objetos, subir un archivo con el mismo nombre no hace duplicados o errores, sino que crea una nueva versión del objeto almacenado. De esta manera se simplifica el flujo de código, permitiendo no añadir comprobaciones explícitas.

La correcta subida de los archivos puede verificarse tanto mediante la consola web de AWS como a través de la línea de comandos. En la Figura 15 se muestra un ejemplo del listado del contenido de un mes concreto dentro de un bucket anual, donde se observa el archivo CSV correctamente almacenado.

```
(cripto_venv) sergio@SergioFdz:~/BIGD/ADAQuant$ aws s3 ls s3://adaquant-2024/month=07/
2026-01-24 19:27:15          0 .keep
2026-01-24 19:27:18       2560 adausdt_daily_2024_07.csv
```

Figura 15. Listado del contenido de un mes específico en un bucket anual de Amazon S3

2.5. Pruebas realizadas

3. Resultados

3.1. Descripción

3.2. Ejecución

3.3. Discusión de los resultados

4. Conclusión

4.1. Resumen del proceso

4.2. Principales logros